

Array (Liste come array)

Array (Liste come array)

Come usare le liste Python come array e quando usare librerie specifiche.

Perché è utile usare gli array in Python

In altri linguaggi come C++ o Java, un **array** è una sequenza di elementi tutti dello stesso tipo, con una posizione numerata che parte da 0. In Python, le **liste** svolgono questo ruolo nella maggior parte dei casi — e sono ancora più flessibili.

Le liste come array

Per uso generico, le liste Python si comportano esattamente come array:

```
# Una sequenza di numeri interi, con indici 0, 1, 2, 3, 4
numeri = [10, 20, 30, 40, 50]

# Accedere a un elemento tramite il suo indice (la posizione)
print(numeri[0])    # 10 ← primo elemento
print(numeri[2])    # 30 ← terzo elemento
print(numeri[-1])   # 50 ← ultimo elemento (si conta da destra)

# Modificare un elemento
numeri[1] = 25
print(numeri)       # [10, 25, 30, 40, 50]
```

Il modulo `array` : più efficiente per grandi quantità di numeri

Python ha un modulo chiamato `array` che offre array più efficienti in memoria rispetto alle liste normali. A differenza delle liste, però, possono contenere solo elementi dello **stesso tipo**.

```
# Crea un array di interi ('i' = integer)
numeri = array.array('i', [1, 2, 3, 4, 5])

print(numeri)          # array('i', [1, 2, 3, 4, 5])
print(numeri[0])        # 1

numeri.append(6)        # Aggiunge un elemento in fondo
print(numeri)           # array('i', [1, 2, 3, 4, 5, 6])

numeri.insert(2, 99)    # Inserisce 99 alla posizione 2
print(numeri)           # array('i', [1, 2, 99, 3, 4, 5, 6])

numeri.remove(99)       # Rimuove la prima occorrenza di 99
print(numeri)           # array('i', [1, 2, 3, 4, 5, 6])
```

I codici tipo più usati:

Codice	Tipo	Descrizione
'i'	int	Numero intero
'l'	long	Numero intero lungo
'f'	float	Numero decimale (singola precisione)
'd'	double	Numero decimale (doppia precisione)

Matrici: array a due dimensioni

Una **matrice** è come una tabella con righe e colonne. In Python si rappresenta come una lista di liste:

```

# Una tabella 3x3
matrice = [
    [1, 2, 3], # riga 0
    [4, 5, 6], # riga 1
    [7, 8, 9]  # riga 2
]

# Accedere a un elemento: prima la riga, poi la colonna
print(matrice[0][0]) # 1 ← riga 0, colonna 0
print(matrice[1][2]) # 6 ← riga 1, colonna 2
print(matrice[2])    # [7, 8, 9] ← tutta la riga 2

# Modificare un elemento
matrice[1][1] = 99
print(matrice[1])    # [4, 99, 6]

```

Scorrere un array (o una matrice) con **for**

```

numeri = [10, 20, 30, 40, 50]

# Scorrere tutti gli elementi
for n in numeri:
    print(n)

# Scorrere con l'indice
for i in range(len(numeri)):
    print(f"numeri[{i}] = {numeri[i]}")

# Scorrere una matrice riga per riga, elemento per elemento
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for riga in matrice:
    for elemento in riga:
        print(elemento, end=" ") # Stampa sulla stessa riga
    print() # Va a capo dopo ogni riga

```

Operazioni comuni

```
numeri = [5, 2, 8, 1, 9, 3]

print(max(numeri))    # 9 – il valore più grande
print(min(numeri))    # 1 – il valore più piccolo
print(sum(numeri))    # 28 – la somma di tutti
print(len(numeri))    # 6 – quanti elementi ci sono
print(sorted(numeri)) # [1, 2, 3, 5, 8, 9] – copia ordinata
```

Quando usare NumPy invece delle liste?

Le liste Python vanno bene per la maggior parte degli usi. Ma se devi fare **calcoli matematici su grandi quantità di numeri** — come la media di 10.000 valori, moltiplicare matrici, o applicare funzioni statistiche — usa la libreria **NumPy**.

NumPy fa le stesse cose delle liste, ma è molto più veloce per i calcoli numerici intensivi. Lo vedremo nella sezione dedicata all'ecosistema Python.